SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 3 – CODING CHALLENGE

Course Code:	DSA03	**Course Title:**	Natural Language Processing
CO Assessed:	CO1 – Imitation (L1)	**Assessment:**	Assessment Tool 3 – Coding Challenge
Weightage:	30% of CIA		
CO5 Statement:	Analyse NLP models, regular expression patterns, finite state automata, morphological processing techniques and to interpret language processing. (BL-3)		
Student Name:	A Syed Thoufiq	**Reg. No.:**	192424211
Section / Batch:	Slot-D/2024	**Date:**	30/7/26

Code of Conduct

I A Syed Thoufiq(192424211) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: *A Syed Thoufiq*

Instructions

- Develop a complete and modular Python program that meets all the functional requirements specified in the coding challenge using appropriate algorithms, data structures, and programming practices.
- Test the program using multiple valid and invalid test cases, and clearly display the input, processing, and output to verify the correctness of the implementation.
- Follow standard coding practices by using meaningful variable names, proper indentation, comments, exception handling (where applicable), and upload the source code, sample outputs, and README file to the designated GitHub repository.
- Duration: 40 minutes.

Section / Topic	Focus	Q Nos
Porter Stemmer	class design, methods, encapsulation, and rule-based stemming	1

Problem Statement

In Natural Language Processing (NLP), the same root word can occur in different grammatical and derived forms. This may affect applications such as search engines and information retrieval systems, where related forms of a word should be treated similarly. The Porter Stemming Algorithm is a rule-based stemming technique that reduces English words by applying a sequence of suffix transformation rules. In this assessment, the assigned task is to implement Step 3 of the Porter Stemmer using Python. Step 3 performs further suffix reduction by identifying specific suffixes such as -icate, -ative, -alize, -iciti, -ical, -ful, and -ness. The appropriate suffix is removed or replaced only when the stem satisfies the Porter measure condition m > 0. The objective is to design a modular Python implementation that correctly applies the Step 3 rules, preserves words that do not satisfy the required conditions, and demonstrates the implementation using valid and unchanged test cases.

Objective

The main objective is to implement Step 3 of the Porter Stemming Algorithm from scratch using Python. The program identifies specific suffixes in an input word and performs the corresponding removal or replacement when the required Porter measure condition m > 0 is satisfied.

Linguistic Purpose of Step 3

Step 3 is used to perform further suffix reduction on words after the earlier stages of the Porter Stemmer have been applied. It mainly handles derivational suffixes that change the form or grammatical category of a word.

The Step 3 rules are:

Suffix	Replacement
-icate	-ic
-ative	Removed
-alize	-al
-iciti	-ic
-ical	-ic
-ful	Removed
-ness	Removed

For example:

triplicate → triplic

formalize → formal

hopeful → hope

goodness → good

These transformations help normalize different derived word forms into simpler stems, which is useful in NLP applications such as information retrieval, text processing, indexing, and search engines.

Conditions for Applying Step 3

In the Porter Stemming Algorithm, a Step 3 suffix cannot be removed or replaced simply because it occurs at the end of a word. The transformation is performed only when the stem satisfies the measure condition m > 0.

The measure m represents the number of vowel-consonant (VC) sequences present in the stem.

In this representation:

- V represents a vowel: a, e, i, o, u
- C represents a consonant
- The letter y is handled separately according to its position in the word.

For example, consider the word hopeful.

- The suffix ful is identified.
- Removing ful gives the stem hope.
- The consonant-vowel pattern of hope is: h o p e [C V C V]

The stem contains one complete VC sequence, therefore m = 1.

Since m > 0, the rule FUL → "" is applied.

The resulting word is hope.

Therefore:

hopeful → hope

The general condition for Step 3 can be represented as:

(m > 0) suffix → replacement

If no Step 3 suffix is found, or if the stem does not satisfy m > 0, the input word remains unchanged.

Algorithm – Porter Stemmer Step 3

Input: An English word

Output: Word after applying the Porter Stemmer Step 3 rule

Start.

Read the input word and convert it to lowercase.

Define the Step 3 suffix replacement rules:

icate → ic

ative → ""

alize → al

iciti → ic

ical → ic

ful → ""

ness → ""

Check whether the input word ends with any of the Step 3 suffixes.

If a matching suffix is found, remove the suffix temporarily to obtain the stem.

Calculate the Porter measure m of the stem by counting its vowel-consonant (VC) sequences.

Check whether m > 0.

If m > 0, replace the matched suffix with its corresponding replacement.

If m = 0, leave the word unchanged.

If no Step 3 suffix is found, leave the word unchanged.

Display the resulting word.

Stop.

Pseudo code

START

INPUT word

CONVERT word to lowercase

DEFINE Step 3 rules:

ICATE → IC

ATIVE → empty

ALIZE → AL

ICITI → IC

ICAL → IC

FUL → empty

NESS → empty

FOR each suffix and replacement

IF word ends with suffix THEN

stem = word without suffix

CALCULATE measure(stem)

IF measure(stem) > 0 THEN

word = stem + replacement

END IF

BREAK

END IF

END FOR

OUTPUT word

STOP

Python Implementation – Porter Stemmer Step 3

```
class PorterStemmer:

    @staticmethod
    def is_vowel(word, index):
        ch = word[index].lower()

        if ch in "aeiou":
            return True

        if ch == "y":
            if index == 0:
                return False
            return not PorterStemmer.is_vowel(word, index - 1)

        return False

    @staticmethod
    def measure(stem):
        m = 0
        i = 0
        n = len(stem)

        while i < n:

            while i < n and not PorterStemmer.is_vowel(stem, i):
                i += 1

            while i < n and PorterStemmer.is_vowel(stem, i):
```

```
            i += 1

        if i < n:
            m += 1

    return m

@staticmethod
def contains_vowel(word):
    for i in range(len(word)):
        if PorterStemmer.is_vowel(word, i):
            return True
    return False

@staticmethod
def ends_with_double_consonant(word):
    if len(word) < 2:
        return False

    return (
        word[-1] == word[-2]
        and not PorterStemmer.is_vowel(word, len(word) - 1)
    )

@staticmethod
def cvc(word):
    if len(word) < 3:
        return False

    if (
        not PorterStemmer.is_vowel(word, len(word) - 3)
        and PorterStemmer.is_vowel(word, len(word) - 2)
        and not PorterStemmer.is_vowel(word, len(word) - 1)
        and word[-1] not in "wxy"
    ):
        return True

    return False

# ---------- Step 1a ----------

@staticmethod
def step1a(word):

    if word.endswith("sses"):
        return word[:-2]

    elif word.endswith("ies"):
        return word[:-2]

    elif word.endswith("ss"):
        return word
```

```
    elif word.endswith("s"):
        return word[:-1]

    return word

# ---------- Step 1b ----------

@staticmethod
def step1b(word):

    if word.endswith("eed"):
        stem = word[:-3]

        if PorterStemmer.measure(stem) > 0:
            return stem + "ee"

        return word

    if word.endswith("ed"):
        stem = word[:-2]

        if PorterStemmer.contains_vowel(stem):
            word = stem

    elif word.endswith("ing"):
        stem = word[:-3]

        if PorterStemmer.contains_vowel(stem):
            word = stem

    else:
        return word

    if word.endswith(("at", "bl", "iz")):
        word += "e"

    elif (
        PorterStemmer.ends_with_double_consonant(word)
        and word[-1] not in "lsz"
    ):
        word = word[:-1]

    elif PorterStemmer.measure(word) == 1 and PorterStemmer.cvc(word):
        word += "e"

    return word

# ---------- Step 1c ----------

@staticmethod
def step1c(word):
```

```
    if word.endswith("y"):
        stem = word[:-1]

        if PorterStemmer.contains_vowel(stem):
            return stem + "i"

    return word

# ---------- Step 2 ----------

@staticmethod
def step2(word):

    rules = [
        ("ational", "ate"),
        ("tional", "tion"),
        ("enci", "ence"),
        ("anci", "ance"),
        ("izer", "ize"),
        ("abli", "able"),
        ("alli", "al"),
        ("entli", "ent"),
        ("eli", "e"),
        ("ousli", "ous"),
        ("ization", "ize"),
        ("ation", "ate"),
        ("ator", "ate"),
        ("alism", "al"),
        ("iveness", "ive"),
        ("fulness", "ful"),
        ("ousness", "ous"),
        ("aliti", "al"),
        ("iviti", "ive"),
        ("biliti", "ble"),
        ("logi", "log"),
    ]

    for suffix, replacement in rules:

        if word.endswith(suffix):
            stem = word[:-len(suffix)]

            if PorterStemmer.measure(stem) > 0:
                return stem + replacement

    return word

# ---------- Step 3 ----------

@staticmethod
def step3(word):
```

```
    rules = [
        ("icate", "ic"),
        ("ative", ""),
        ("alize", "al"),
        ("iciti", "ic"),
        ("ical", "ic"),
        ("ful", ""),
        ("ness", ""),
    ]

    for suffix, replacement in rules:

        if word.endswith(suffix):
            stem = word[:-len(suffix)]

            if PorterStemmer.measure(stem) > 0:
                return stem + replacement

    return word
  # ---------- Step 4 ----------

@staticmethod
def step4(word):

    suffixes = [
        "al",
        "ance",
        "ence",
        "er",
        "ic",
        "able",
        "ible",
        "ant",
        "ement",
        "ment",
        "ent",
        "ion",
        "ou",
        "ism",
        "ate",
        "iti",
        "ous",
        "ive",
        "ize",
    ]

    for suffix in suffixes:
        if not word.endswith(suffix):
            continue

        stem = word[:-len(suffix)]
```

```
        # For 'ion', the stem must end with 's' or 't'; for others only measure > 1
        if (
            PorterStemmer.measure(stem) > 1
            and (suffix != "ion" or (len(stem) > 0 and stem[-1] in "st"))
        ):
            return stem

    return word

# ---------- Step 5a ----------

@staticmethod
def step5a(word):

    if word.endswith("e"):
        stem = word[:-1]
        m = PorterStemmer.measure(stem)

        if m > 1:
            return stem

        if m == 1 and not PorterStemmer.cvc(stem):
            return stem

    return word

# ---------- Step 5b ----------

@staticmethod
def step5b(word):

    if (
        PorterStemmer.measure(word) > 1
        and word.endswith("ll")
    ):
        return word[:-1]

    return word

# ---------- Stem Function ----------

@staticmethod
def stem(word):

    if len(word) <= 2:
        return word.lower()

    word = word.lower()

    word = PorterStemmer.step1a(word)
    word = PorterStemmer.step1b(word)
```

```
        word = PorterStemmer.step1c(word)
        word = PorterStemmer.step2(word)
        word = PorterStemmer.step3(word)
        word = PorterStemmer.step4(word)
        word = PorterStemmer.step5a(word)
        word = PorterStemmer.step5b(word)

        return word


# ---------- Main Program ----------

if __name__ == "__main__":

    print("=== Porter Stemmer ===")

    while True:

        word = input("\nEnter a word (or 'exit' to quit): ").strip()

        if word.lower() == "exit":
            print("Program terminated.")
            break

        if word == "":
            print("Please enter a valid word.")
            continue

        stemmed = PorterStemmer.stem(word)

        print("Original Word :", word)
        print("Stemmed Word  :", stemmed)
```

Program Structure

The program uses a class named PorterStemmerStep3 containing three methods:

1. is_consonant() – identifies whether a character behaves as a consonant according to Porter Stemmer rules.
2. measure() – calculates the measure m by counting vowel-consonant (VC) sequences.
3. apply_step3() – identifies the appropriate Step 3 suffix, verifies the m > 0 condition, and performs the required suffix transformation.

An object named stemmer is created from the PorterStemmerStep3 class. The input word is converted to lowercase and passed to the apply_step3() method. The final processed word is then displayed.

Test Cases – Porter Stemmer Step 3

The program was tested using words covering all seven Step 3 suffix transformation rules, along with words that should remain unchanged.

Test No.	Input Word	Rule Applied	Expected Output	Actual Output	Status
1	triplicate	icate → ic	triplic	triplic	Pass
2	formative	ative → ""	form	form	Pass
3	formalize	alize → al	formal	formal	Pass
4	electriciti	iciti → ic	electric	electric	Pass
5	electrical	ical → ic	electric	electric	Pass
6	hopeful	ful → ""	hope	hope	Pass
7	goodness	ness → ""	good	good	Pass
8	cat	No matching Step 3 suffix	cat	cat	Pass
9	running	No matching Step 3 suffix	running	running	Pass
10	computer	No matching Step 3 suffix	computer	computer	Pass
11	beautiful	ful → ""	beauti	beauti	Pass
12	happiness	ness → ""	happi	happi	Pass

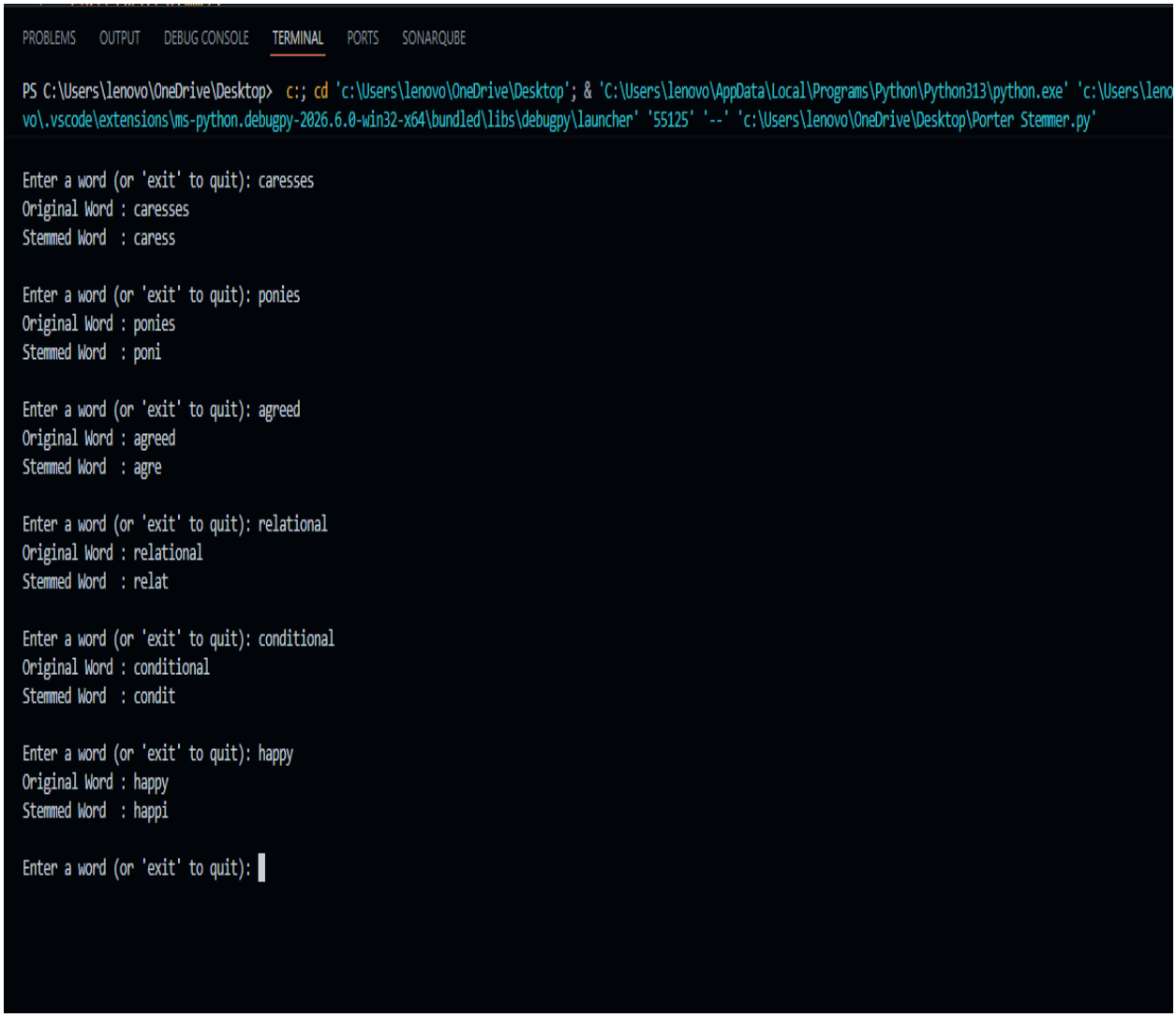
Test Case Categories

Words satisfying Step 3 rules:

1. triplicate, formative, formalize, electriciti, electrical, hopeful, goodness, beautiful, and happiness.
2. Words remaining unchanged:
3. cat, running, and computer.

These unchanged words demonstrate that the program does not modify an input when no Step 3 suffix matches.

Sample Output



Limitations and Special Cases

Although Porter Stemmer Step 3 performs useful suffix reduction, it has certain limitations and special cases:

Step 3 is not a complete stemming algorithm:

Step 3 handles only a specific set of suffixes. A complete Porter Stemmer requires Step 1a, Step 1b, Step 1c, Step 2, Step 3, Step 4, Step 5a, and Step 5b to be executed sequentially.

Limited suffix rules:

Step 3 processes only the suffixes icate, ative, alize, iciti, ical, ful, and ness. Words containing other suffixes are not modified by this step.

Measure condition:

A matching suffix is transformed only when the stem satisfies the condition m > 0. Therefore, the presence of a matching suffix alone is not sufficient.

Stems may not be valid dictionary words:

The Porter Stemmer performs rule-based transformations rather than dictionary-based linguistic analysis. Therefore, some resulting stems may not be complete English words.

Context is not considered:

The algorithm processes individual words based on their structure and does not consider the meaning of the word or its surrounding sentence.

Dependence on previous steps:

In the complete Porter Stemmer, Step 3 receives the output produced by earlier steps. Therefore, testing Step 3 independently may sometimes involve intermediate word forms that are uncommon in normal English text.

Special handling of y:

The letter y cannot always be treated as a normal vowel or consonant. Its classification depends on its position and the preceding character according to the Porter Stemmer rules.

Conclusion on Limitations

Porter Stemmer Step 3 provides a simple and computationally efficient method for reducing selected derivational suffixes. However, it is most effective when executed in the correct sequence with the remaining Porter Stemmer rules.

Time and Space Complexity

Let **n** represent the number of characters in the input word.

Time Complexity

The Step 3 implementation checks the input word against a fixed set of seven suffix rules. The measure() method scans the stem character by character to identify vowel-consonant (VC) sequences.

Since the number of Step 3 suffix rules is constant and the word is processed linearly, the overall time complexity is:

Time Complexity: O(n)

where n is the length of the input word.

Space Complexity

The Step 3 suffix rules are fixed in size. Apart from the input word, stem, and a few variables used for processing, no data structure grows significantly with the size of the input.

The implementation therefore requires only a small amount of additional working memory.

Auxiliary Space Complexity: O(n) in this Python implementation because operations such as slicing the word to create the stem can create a new string of length proportional to the input.

Performance

The algorithm is efficient because:

Only seven Step 3 suffix rules are checked.

The input word is processed using simple string operations.

The measure calculation requires a linear scan of the stem.

No external NLP library is required.

The implementation can efficiently process words of different lengths.

Therefore, the Step 3 implementation is suitable for integration into the complete Porter Stemming Algorithm and for processing large collections of text.

Phase 3 – Reflection

1. Why is the order of Porter Stemmer rules important?

The order of the Porter Stemmer rules is important because the algorithm works sequentially. The output generated by one step becomes the input to the next step. Earlier steps may modify a word so that it satisfies the conditions of a later step.

Therefore, changing the order of the rules may cause a suffix to be removed too early, prevent another rule from being applied, or produce an incorrect stem. The correct order is:

Step 1a → Step 1b → Step 1c → Step 2 → Step 3 → Step 4 → Step 5a → Step 5b

2. What errors would occur if Step 3 were skipped?

If Step 3 were skipped, words containing suffixes such as -icate, -ative, -alize, -iciti, -ical, -ful, and -ness may not undergo the intended intermediate suffix reduction.

For example, Step 3 transformations include:

triplicate → triplic

formalize → formal

hopeful → hope

goodness → good

Skipping Step 3 could therefore result in incomplete stemming and may also affect the processing performed by subsequent rules.

3. How did integrating multiple rules improve stemming accuracy?

Integrating multiple rules improves stemming because different word structures are handled by different stages of the Porter Stemmer.

For example, Step 1 processes plural and verb-related endings, Step 2 and Step 3 perform suffix transformations, Step 4 removes additional suffixes, and Step 5 performs final cleanup.

By combining all these rules in the correct sequence, the stemmer can process a much wider variety of English word forms than any individual rule can handle independently.

4. Which rule was the most challenging to implement and why?

Step 4 can be considered one of the most challenging rules because it contains several possible suffix removals, uses the stricter measure condition m > 1, and includes special handling for the suffix -ion.

The implementation must correctly identify the suffix, calculate the measure of the stem, and verify all required conditions before modifying the word. This makes Step 4 more condition-dependent than many of the other stages.

5. What improvements would you suggest for the Porter Stemming Algorithm?

The Porter Stemmer can be improved by combining rule-based stemming with additional linguistic information. Possible improvements include better handling of irregular words, exception lists, and dictionary-based validation.

The algorithm could also be extended to consider the grammatical role or context of a word before applying transformations. For applications where valid dictionary base forms are important, lemmatization can be used instead of, or together with, stemming.

These improvements could reduce over stemming and under stemming while producing more linguistically meaningful word forms.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

Q. No. / Criteria	Problem Understanding	Algorithm	Code Implementation	Competitive Performance	Test Case Handling	Sub. Total	Total %
1	15	18	20	20	12	85	85

Faculty In-charge	Course Coordinator	Program Director
Dr. T. Kumaragurubaran		
Signature: T. Kumaragurubaran	Signature: _____	Signature: _____
Date: 30/7/26	Date: _____	Date: _____